

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
-2 ** 2	-4	erro de sintaxe	(não há **)
2 ** 3 ** 2	512	512	-
1 + 2 + "3"	TypeError	"33"	"33"
-7 / 2 (inteiros)	-4 (com //)	-3.5	-3
-7 % 2	1	-1	-1

Associatividade (p. 303)
Esquerda: * / % + -
Direita: ** (Python, JS, Ruby, Fortran), ++, - -, menos unário
Visual Basic: ^ à esquerda!

Excel
=-2^2 dá 4: lá o menos unário vence o ^.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Entendi que a ordem em que uma expressão é calculada depende da precedência e associatividade dos operadores

Slide 7

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. · §7.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

A ordem em que os valores são avaliados pode mudar o resultado quando uma função altera alguma variável, tipo variáveis globais e locais

Slide 8

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);           // 98
System.out.println("" + 'a' + 1);     // a1
x.add(y).multiply(tres);               // BigDecimal:
sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):           # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):               # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)         # Vetor(4, 6)
print(v * 3)                          # Vetor(12, 18)
```

Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

Um mesmo operador, como +, pode ter diferentes funções dependendo dos valores utilizados

Slide 9

Conversões de tipos: alargamento, estreitamento e coerção

Java • coerção e cast

```
int grande = 123456789;
float f = grande;          // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. • §7.4 • p. 310-313 • exemplos/04_conversoes

Entendi a diferença entre alargamento, estreitamento, coerção e cast, algumas conversões acontecem automaticamente, enquanto outras precisam ser feitas manualmente para evitar perda de dados.

Slide 10

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + +"a" + "a" // "baNaNaN"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP • "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01");        // true

md5("240610708") // "0e462097431906..."
md5("QNKCDZO")   // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com === e !==, que **não** fazem coerção.

Sebesta, 11ª ed. • §7.4.1 Coerção em expressões • p. 311-312 • exemplos/04_conversoes

Foi possível entender que JavaScript e PHP podem converter tipos automaticamente durante operações e comparações, isso acaba por facilitar algumas coisas, mas também pode gerar resultados inesperados.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0    0.0    ""    []    {}    None
# contam como VERDADEIRO:
# "0"    [0]    " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Diferentes linguagens interpretam valores verdadeiros e falsos, também vi que comparações encadeadas podem funcionar de maneira diferente dependendo da linguagem.

Slide 12

Avaliação em curto-circuito

Java • && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;          // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    //
NullPointerException
```

JavaScript • ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade           // undefined, sem erro

const qtd = 0;
qtd || 10                    // 10  (o 0 se perdeu!)
qtd ?? 10                    // 0   (só troca
null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve `"padrão"`: `and` e `or` devolvem um dos **operandos**.

Sebesta, 11ª ed. • §7.6 • p. 315-317 • exemplos/05_booleans

Apreendi que operadores como `&&` e `||` podem evitar que partes desnecessárias de uma expressão sejam executadas, isso também pode ajudar a evitar erros, como acessar um valor nulo.

Sentenças de atribuição

C • a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do
livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }          // compila! atribui e
testa y
```

Python • atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem
temporária
if (n := len(dados)) > 5: ... # morsa (Python
3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Linguagens funcionais (p. 321)

val (ML) e let (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. • §7.7 • p. 317-321 • exemplos/06_atribuicao

A atribuição de valores não funciona exatamente igual em todas as linguagens, em algumas ela pode fazer parte de uma expressão, enquanto outras possuem regras mais restritas.

Slide 14

Atribuição de modo misto

Java • só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;       // 3 (cast explícito)
long l = i;              // ok

i += 3.7;                // COMPILA: cast
                          // escondido
System.out.println(i);   // 6
```

C • coerção em qualquer direção

```
int i = 3.7;             // 3, em silêncio
char c = 300;            // 44 (só um aviso)
unsigned int u = -1;      // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para `double`.

Sebesta, 11ª ed. • §7.8 • p. 321-322 • exemplos/06_atribuicao

Foi possível entender que atribuir valores de tipos diferentes pode causar conversões e até perda de informação, também é importante tomar cuidado com operações entre inteiros antes de armazenar o resultado em um `double`.

Slide 15

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;  
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...  
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)  
  x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. - §8.1 - p. 328-330

Entendi que sequência, seleção e repetição são as estruturas básicas necessárias para construir algoritmos

Seleção de dois caminhos: o "else pendente"

C • a indentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    // de qual if é este
else?
    result = 1;
printf("%d\n", result);    // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A indentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };
p. 334)
y = x if x > 0 else 2 * x
```

// Rust: if é uma expressão (como em F#,

Python: expressão condicional (p. 306)

Apreendi que a indentação nem sempre define a qual if um else pertence, usar chaves deixa o código mais claro e evita erros causados por interpretações erradas.

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {  
  case 1:  
    case 3: odd += 1;  
    case 2:  
    case 4: even += 1;  
    default: printf("Error\n");  
}  
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {  
  case SAB, DOM -> "fim de  
semana";  
  case SEG, TER, QUA,  
    QUI, SEX -> "dia útil";  
}; // sem break, sem cair;  
// exaustivo para enum
```

Rust · match exaustivo

```
match n {  
  i32::MIN..=-1 => "negativo",  
  0 => "zero",  
  1..=9 => "um dígito",  
  _ => "vários dígitos",  
}  
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige fallthrough. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Entendi como estruturas como switch e match escolhem entre vários caminhos, algumas linguagens evitam automaticamente problemas como executar vários casos por esquecer o break.

Slide 18

Laços controlados por contador

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
                        // 0 3 6 9  
  
for (; i <= 100; ) // expressões  
opcionais  
    soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                        # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. · §8.3.1 · p. 342-345 · exemplos/08_iteracao

Foi possível entender que o for pode funcionar de maneiras diferentes entre as linguagens, em C alterar o contador dentro do laço afeta a repetição, enquanto no for com range do Python funciona diferente.

Slide 19

Laços lógicos, saídas e iteradores

Java • break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
    for (int j = 0; j < m[i].length; j++)
        if (m[i][j] < 0) {
            System.out.println(i + "," + j);
            break externo;    // sai dos DOIS
    }
    laços
```

JavaScript • for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total);    // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total);    // 60: valores
```

Zune (2008) • o laço lógico sem progresso

```
while (dias > 365) {
    if (bissexto(ano)) {
        if (dias > 366) { dias -= 366; ano++; }
        // dias == 366: nada muda, laço infinito!
    } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. • §8.3.2-8.3.4 • p. 345-352 • exemplos/08_iteracao

Entendi melhor o funcionamento de while, break, iteradores e diferentes tipos de for, também vi como um laço que não faz progresso pode acabar ficando infinito.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) • versão simplificada

```
int valida_certificado(void) {  
    int err;  
    if ((err = verifica_hash()) != 0)  
        goto fail;  
        goto fail;           // duplicada: desvia  
SEMPRE  
    if ((err = verifica_assinatura()) != 0)  
        goto fail;           // nunca é alcançada  
fail:  
    return err;               // 0 = "certificado  
válido"  
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. • §8.4 • p. 352-353 • exemplos/09_desvio_protegido

Aprendi que o goto permite alterar diretamente o fluxo do programa, mas pode deixar o código difícil de entender e causar erros, estruturas como break, continue e return são alternativas mais controladas.

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x == y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no `if`) ou fim do laço (no `do ... od`)
- Base do **CSP** de Hoare (1978) e do `select` de Go

Go • `select`, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada
// execução)
```

Sebesta, 11ª ed. • §8.5 • p. 353-355 • exemplos/09_desvio_protegido

Entendi que comandos protegidos trabalham com várias condições possíveis e podem escolher uma delas quando mais de uma for verdadeira, essa ideia também influenciou recursos modernos, como o `select` do Go.